Analyzing the Behavior of LLM Under Concurrency and Token-Based DoS Attacks

MD Abdul Barek, A.B.M Kamrul Islam Riad
Department of Intelligent Systems and Robotics
University of West Florida
Pensacola, USA
e-mail: {mb381, ai62}@students.uwf.edu

Md Bajlur Rashid
Department of Cybersecurity and Information Technology
University of West Florida
Pensacola, USA
e-mail: mr248@students.uwf.edu

Guillermo Francia III, Hossain Shahriar
Center for Cybersecurity
University of West Florida
Pensacola, USA
e-mail: {gfranciaiii, hshahriar}@uwf.edu

Sheikh Iqbal Ahamed
Department of Computer Science
Marquette University
Milwaukee, USA
e-mail: sheikh.ahamed@mu.edu

2025 IEEE Conference on Dependable, Autonomic and Secure Computing (DASC) | 979-8-3315-7045-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/DASC68382.2025.00017

***Abstract*—A Large Language Model (LLM) is an AI system that uses deep learning and extensive data to comprehend, process, and produce human-like language. Globally, LLMs have shown promising results and have gained widespread acceptance. However, this widespread adoption and numerous applications of LLMs have simultaneously heightened their appeal as targets for malicious attackers. Attackers continuously develop new strategies to disrupt computer systems and also attack LLM in many ways. In addition, LLMs have intrinsic vulnerabilities and limitations that can be leveraged and exposed to a range of cyberattacks. Among these, Denial of Service (DoS) is a prevalent threat that disrupts the functionality of LLMs by inundating them with excessive input or even making them completely unavailable. In addition to traditional concurrent-based DoS attacks, we also explore a novel prompt-based attack, where repeating tokens within a single input prompt can internally overload the LLM and trigger failure, even without concurrency. This paper explores how LLMs react to such attacks, demonstrating their behavior under varying levels of DoS attacks, and comparing the outcomes. It also investigates how repeated token prompt structures can cause instability, revealing a new class of input-driven DoS vulnerabilities in LLMs. Our experiments on three open-source LLMs confirm that both high-concurrency and repeated token inputs can significantly degrade performance, increase response time, and even lead to system crashes under high load. Furthermore, to address the concurrency-based DoS attack, we have implemented and validated a queue-based mitigation approach in our companion work.**

***Index Terms*—Artificial Intelligence(AI), DoS attack, Large Language Model(LLM), LLM behaviour, Tokenization, Input-Level Attacks.**

I. Introduction

Large Language Models (LLMs) are advanced AI systems capable of generating and understanding text and enables coherent communication and adaptability in a wide range of tasks[1]. One may regard an LLM as a sophisticated computational model trained on extensive datasets and allows it to comprehend and process human language or other intricate forms of data. LLMs have gained immense popularity in different domains due to their impressive performance and promising results [2]. It has already reached a significant milestone of Natural Language Processing (NLP), showing unprecedented and innovative capabilities generating human-like texts in recent years [3]. LLMs inherently encompass a variety of vulnerabilities and limitations that arise from their architectural design, training data, and algorithms. They are constrained by the quality of the data they ingest. When supplied with erroneous information, they produce inaccurate outputs in response to user queries. Additionally, LLMs may experience ”hallucinations,” [4] in which details are fabricated when unable to derive a precise solution. These models can also be exploited via maliciously crafted inputs. Coercing them to generate biased, unethical, or even hazardous responses. According to OWASP [5], there are LLM vulnerabilities that show the most important security issues when using large language models (LLMs). These threats include prompt injection[6], where harmful inputs manipulate LLM to give incorrect or unsafe responses, and insecure output handling[7], where unchecked LLM responses can cause security issues. According to the 2025 OWASP, several vulnerabilities have been reclassified. For example, Denial of Service has been renamed Unbounded Consumption, and Insecure Output Handling is now called Improper Output Handling [8]

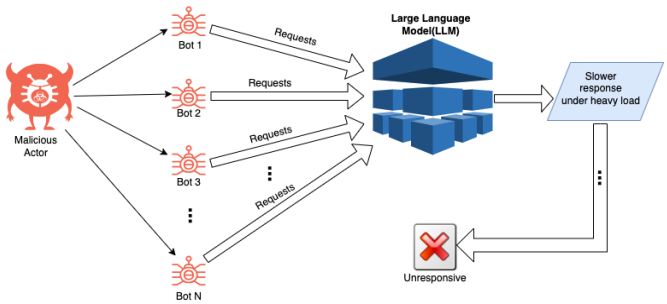

Fig. 1: LLM under DoS attack.

2837-0740/25/$31.00 ©2025 IEEE
DOI 10.1109/DASC68382.2025.00017

There is also training data poisoning[9], which happens when bad training data makes the model give inaccurate results. Other risks include supply chain vulnerabilities[10], where outside components could add dangerous code, and sensitive information disclosure, where private data could be accidentally shared. Poorly designed LLM plugins[11] may be vulnerable in Large Language Model (LLM) systems that arises when plugins—extensions that enhance LLM functionality—are not properly secured. Excessive agency[12] in LLM-based systems can occur when the model is given too much control or access to perform actions without enough restrictions. Over-reliance[13] may occur when users or systems place excessive trust in the output of LLMs without proper verification or oversight.Furthermore, model theft[14] is a vulnerability where attackers replicate proprietary Large Language Models (LLMs) without authorization.

A particularly disruptive vulnerability is the Denial of Service (DoS) attack [15, 16], a special type of cyberattack that aims to make a computer resource or service unavailable to its intended users. The main goal of a DoS attack is to disrupt a service by overwhelming it with a flood of illegitimate requests, causing the system to become unresponsive or completely fail. Such attack is especially harmful to software, APIs, or plugins that are integral to an accurate LLM outcome [7].

In addition to these concurrency-based DoS threats, this paper introduces and analyzes a novel input-level attack: the Token-Based Prompt-Induced DoS. In this method, a single input prompt is crafted by repeating tokens—such as a sentence or word—until it reaches the model's maximum token length. Even though only one request is made, this input can internally overwhelm the model's memory and attention mechanisms, leading to delays, system crashes, or failures. This form of attack does not rely on user concurrency and is capable of bypassing rate-limiting protections[17], making it a stealthier yet highly effective strategy for destabilizing LLMs.

The main contribution of this paper is to demonstrate graphically and quantitatively how LLMs behave under a DoS attack and compare the performance of various open-source LLM services under this specific attack. The DoS attack on LLM is illustrated in Figure 1, where a malicious actor launches numerous inquiries to the LLM service using N number of bots. These bots generate multiple concurrent inquiries, intentionally overwhelming the LLM service. Initially, the LLM may respond well, but as the level of attack continues to grow, the LLM service gradually slows down and becomes unresponsive or it may even eventually become unavailable. To address this concurrency-based threat, we have implemented and validated a queue-based mitigation approach, as described in our companion work [18], which demonstrably reduced latency, improved model stability under concurrent load, and effectively mitigated the impact of DoS attacks.

In addition to this concurrency-based study, we also evaluate the impact of single long prompts with repeated tokens on LLMs, demonstrating that internal prompt structure alone—without concurrent load—can cause comparable performance degradation and model instability.

II. Architectural Complexity of LLMs and Their Susceptibility to DoS Attacks

Large Language Models (LLMs) are based on the Transformer architecture, which is inherently complex and computationally demanding. Introduced by Vaswani et al. [19], the Transformer consists of multiple layers of multi-head self-attention and feedforward neural networks, each operating over high-dimensional token embeddings. The self-attention mechanism has a time and memory complexity of $O(n^2)$, where n is the number of tokens in the input sequence. This quadratic growth significantly increases memory usage and processing time as input length increases.

This internal design leads to a high sensitivity to Denial of Service (DoS) vectors. First, concurrent requests strain available compute resources such as CPU cores, RAM, or GPU VRAM. Second, repeated-token prompts can overload attention and decoding layers, leading to response delays or crashes, even without concurrency.

These architectural and computational characteristics underscore the importance of not only evaluating LLM behavior under normal and adversarial load, but also designing robust mitigation strategies that can scale with model size and user demand.

III. Formal Specification of the Problem

Let $\mathcal{M}$ denote a deployed Large Language Model (LLM), exposed through an API that processes input prompts $P = \{p_1, p_2, ..., p_n\}$ and returns responses $R = \{r_1, r_2, ..., r_n\}$ such that $\mathcal{M}(p_i) = r_i$. Each prompt p_i is a sequence of tokens $p_i = \{t_1, t_2, ..., t_k\}$, where k is the token length. Let $\mathcal{T}$ denote the tokenizer, and $\mathcal{C}$ be the compute environment (e.g., CPU, memory resources) serving $\mathcal{M}$.

We define *Concurrency-Based DoS* as a function $A_c : \{p_1, ..., p_n\} \mapsto R$ that maximizes resource exhaustion by issuing n requests in parallel, where $n \gg 1$ and $\sum_{i=1}^{n} \text{Latency}(r_i)$ becomes unbounded or leads to timeout/failure. In contrast, a *Token-Based Prompt-Induced DoS* attack is a function $A_t(p) : p \mapsto \hat{p}$, where $\hat{p}$ is crafted to have high repetition, low entropy, and $\text{len}(\hat{p}) \rightarrow$ max_tokens. This attack aims to degrade or crash $\mathcal{M}$ with a single prompt, i.e., $|\hat{P}| = 1$, while still forcing elevated internal resource usage.

Formally, we define the goal of the adversary as maximizing system degradation $\mathcal{D}$, caused by either a concurrency-based attack A_c or a token-based prompt-induced attack A_t

where:

$$\mathcal{D} = \arg\max_{A} \left[\text{Latency}(\mathcal{M}(A(P))) + \text{Resource\_Usage}(\mathcal{C})\right]$$

subject to constraints: $|A_c(P)| > 1$ for concurrency-based attack $|A_t(P)| = 1$ and $\text{Entropy}(A_t(p)) \rightarrow 0$ for prompt-based attack.

This formulation captures two novel DoS vectors where the adversary does not need to compromise the model or infrastructure, but instead manipulates inputs to overload system behavior.

IV. Types of DoS Attacks on LLMs

In LLM, Denial of Service (DoS) attacks can have significant consequences, especially when exploiting the resource-intensive nature of these models.

TABLE I: Types of Denial of Service (DoS) Attacks on LLMs

DoS Attack Type	Key Characteristics
Flooding Attacks	High volume of requests sent rapidly to overwhelm the system.
Model Exhaustion Attacks	Involves complex or long output requests to overload LLM computation.
Slow-rate Attacks	Keeps many connections open for a long time, using minimal data transfer.
Prompt Injection DoS	Malicious prompts trigger recursive or heavy computations in LLM.
Distributed Denial of Service (DDoS)	Multiple sources attack simultaneously from different locations.

There are various types of DoS attacks, a few are shown in table I, that can be particularly impactful on LLM-based services. Flooding attacks(DoS), where the service is overwhelmed with high volumes of requests leading to degraded response times or total service outages. Model exhaustion attacks [20] which focus on overloading the system's memory or CPU by forcing the LLM to handle resource-heavy tasks. It can make requests, with the intent for long or complex outputs that can overwhelm the LLM's computation capacity. This can be serious particularly in cloud-based environments, which rely on finite resources. Slow-rate attacks, such as Slowloris [21] keeps connections open but incomplete, slowly consuming the available connection pool and preventing legitimate requests from being processed. Other sophisticated attacks include prompt injection DoS [22], where inputs are crafted to trigger harmful recursive operations to exploit the internal prompt-handling mechanisms of LLMs.

Moreover, Distributed Denial of Service (DDoS) [23] attacks amplify the impact of flooding attacks by leveraging multiple sources, often a network of compromised machines (botnet), to send requests simultaneously from different locations. DDoS attacks can significantly degrade the performance of LLM-based services or even make them entirely unavailable as they overwhelm not only the target system but also the supporting infrastructure like networks and load balancers.

The impact of these attacks is not limited to reduced service availability but can also lead to increased operational costs as additional resources might be needed to handle and reduce the effects of the attack. Effective countermeasures, such as rate limiting, resource monitoring, queuing systems, and advanced DDoS mitigation techniques like traffic filtering and anomaly detection, are necessary to ensure the continued availability of LLM-based services.

V. Token-Based Prompt-Induced DoS Attack

Tokenization is a core component in the architecture of Large Language Models (LLMs), serving as the initial step where raw input text is segmented into smaller units called tokens[24]. These tokens—ranging from individual characters to subwords or words—form the basic elements that LLMs use to process and understand language. The effectiveness of an LLM heavily relies on how well this tokenization captures meaningful patterns in the input data. Without proper tokenization, models tend to revert to simple frequency-based predictions, missing deeper semantic relationships. This highlights the importance of designing robust tokenization strategies to improve the learning and generalization abilities of LLMs [25]. Recent study[26] further reveals that tokenization can be exploited through crafted inputs to disrupt LLM performance.

So, in addition to conventional DoS attacks that rely on high volumes of concurrent requests, we identify a novel input-level vulnerability that can degrade the performance of Large Language Models (LLMs) using a single, syntactically valid prompt. This technique, referred to as a token exhaustion attack, involves repeatedly inserting the same word or sentence until the input reaches the model's maximum token limit. As illustrated in figure 2, the attacker sends a prompt

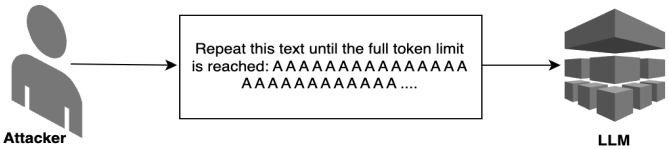

Fig. 2: Token-Based Prompt-Induced DoS Attack

filled with repeated tokens such as "A A A A A…" to exploit the LLM's internal processing mechanisms. Although the input appears harmless, it consumes substantial computational resources, particularly in the model's attention[27] and memory layers[28]. This leads to increased latency, degraded responsiveness, and in some cases, complete system failure. Unlike traditional concurrency-based DoS attacks, this method does not require multiple users or requests and can bypass standard protection mechanisms such as rate limiting. Our experiments also demonstrate that even without parallel input, repeated-token prompts can significantly affect the stability and responsiveness of several widely used open-source LLMs.

VI. Related Works

Wang, et al. [29] introduce ShieldGPT, a novel LLM-based framework for DDoS mitigation that leverages large language models for comprehensive attack analysis and mitigation. They created two specialized role-based prompt templates that generate clear explanations and actionable mitigation instructions, addressing potential hallucination issues. Guastalla, et al. [23] investigate the use of large language models (LLMs), including OpenAI's ChatGPT variants (GPT-3.5, GPT-4, and Ada) to enhance DDoS attack detection. Their study shows that LLMs, particularly when applied in a few-shot learning context or fine-tuned, achieved superior accuracy in detecting DDoS threats.

Mirkovic, et al. [30] discuss the growing problem of Distributed Denial-of-Service (DDoS) attacks and the complexity of both attack strategies and defense mechanisms. Nazario, et al. [31] discuss the evolving nature of Distributed Denial-of-Service (DDoS) attacks, highlighting how attackers use botnets to overwhelm victims with traffic. They explain how DDoS

attacks rely on large-scale bandwidth aggregation through multiple compromised hosts. Robey, et al. [32] introduce SmoothLLM, a novel algorithm designed to mitigate jailbreaking attacks on large language models (LLMs) like GPT, Llama, and Claude. ABM Kamrul, et al. detect security issues in digital apps [33–38] through static code analysis.

Wang, D, et al. [26] investigates a critical vulnerability in Large Language Models (LLMs) related to their tokenization process. The authors observe that incorrect tokenization can lead LLMs to misinterpret inputs, resulting in inaccurate outputs. Yang, J. et al. [39] introduces a new way to improve how Large Language Models (LLMs) break down text into tokens. Current methods like Byte Pair Encoding (BPE) work well but struggle with non-English languages and phrases made of multiple words. They create a new tokenizer called Less-is-Better (LiB), which learns to use a mix of words, subwords, and phrases more efficiently.

These studies provide valuable insights into how large language models(LLMs) can be applied in detecting Denial of Service (DoS) attacks and other security threats. They explained leveraging LLMs for cyber defense tasks, including DoS, DDoS detection, jailbreaking attacks and incident response. They also highlighted the vulnerabilities in LLMs, particularly in adversarial scenarios, and proposed mitigation strategies. However, none of these studies specifically addressed how to mitigate DoS attacks when the Large Language Model itself is the target. Therefore, our research primarily aims to fill this gap by demonstrating LLM(s) behavior under DoS attacks. In addition, we extend our analysis by introducing a token-based prompt-induced DoS attack, a novel input-level threat that can destabilize LLMs using a single repeated-token prompt—demonstrating that even non-concurrent inputs can lead to system failures.

VII. Our Experiments

A. Concurrent requests as a DoS attacks

In order to observe the behavior of the LLM under normal and massive requests, we simulated Denial of Service (DoS) attacks shown in Figure 3. We created a high volume of concurrent requests to the LLM model directly. In our analysis, we initially applied a normal load and saw that the LLM responded well. However, as we gradually increased the load, the response time of the LLM steadily rose. Notably, under heavy load, all of the LLM models responded very slowly, and occasionally, it became almost unresponsive. We took a simple approach to test all the LLM model's performance.

We conducted our experiments on three open-source Large Language Models (LLMs) provided by Hugging Face, focusing on their performance under the Denial of Service (DoS) attack shown in table II. Our first model of choice was the Generative Pre-trained Transformer 2 (GPT-2), a well-known LLM designed to generate human-like text based on input prompts. Secondly, we used BLOOM LLM, which is a 176B-parameter open-access language model created by hundreds of researchers to make large language models (LLMs) accessible. It is trained on data from 59 languages and it performs

```
54  def simulate_requests_for_dos(num_threads):
55      global thread_times
56      threads = []
57      thread_times = []
58      for i in range(num_threads):
59          thread = threading.Thread(target=task_to_llm, name=f"Thread-{i + 1}")
60          threads.append(thread)
61          thread.start()
62      for thread in threads:
63          thread.join()
```

Fig. 3: Simulation of DoS attack using Python threading.

competitively across benchmarks and improves further with multitask fine tuning. To support research and applications, the model and code are released under the Responsible AI License[40]. We chose bigscience/bloom-560m version for our experiment.

TABLE II: Open source LLMs Used in the Experiment

Model	Description	Parameters	Source
GPT-2	A well-known LLM designed to generate human-like text, widely used for natural language processing tasks.	Not specified	Hugging Face
BLOOM (bigscience /bloom-560m)	A 176B-parameter open-access model trained on data from 59 languages, focused on accessibility and competitive benchmarks.	560M	Hugging Face (Responsible AI License)
OPT (facebook/opt-1.3b)	Developed by Meta AI, OPT supports parameter sizes from 125M to 175B, similar in performance to GPT-3.	1.3B	Hugging Face

Finally, we used Open Pre-trained Transformers (OPT), proposed by Meta AI, having parameters supports ranging from 125M to 175B and it performs similar in performance to GPT3[41]. We chose a version of facebook/opt-1.3b for our experimental purpose.

We used Python's multi-threading concept to implement concurrent execution[42]. We provided the same prompt to each model, saying, "Describe the earth in one easy sentence," using a Python script that was set up the same way for each model. In order to generate text from the supplied prompt, we passed exactly the same parameters shown in Figure 5 to those LLM models and followed exactly the same configuration in the Python scripts.

```
40  def task_to_llm():
41      local_start_time = time.time()
42      global thread_times
43      user_prompt = "Describe the earth in one easy sentence."
44      generate_text_with_prompt(model, tokenizer, user_prompt, max_length=150)
45      local_end_time = time.time()
46      execution_time = local_end_time - local_start_time
```

Fig. 4: Call LLM method to generate result

We used the Hugging Face `generate()` function with standard decoding parameters across

```python
def generate_text_with_prompt(model, tokenizer, prompt_text, max_length):
    input_ids = tokenizer.encode(prompt_text, return_tensors="pt")
    output = model.generate(
        input_ids,
        max_length=max_length,
        num_return_sequences=1,
        no_repeat_ngram_size=2,
        pad_token_id=tokenizer.eos_token_id,
        top_k=50,
        top_p=0.95,
        temperature=1.0,
        do_sample=True,
        early_stopping=True
    )
    generated_text = tokenizer.decode(output[0], skip_special_tokens=True)
    return generated_text
```

Fig. 5: LLM result from the given prompt in Python

all models (e.g., `max_length=150`, `top_k=50`, `top_p=0.95`, `temperature=1.0`, `do_sample=True`, `early_stopping=True`) to ensure consistent text generation.

The code snippet of this simulation is shown as algorithm 1. This algorithm is designed to simulate handling multiple requests concurrently by using threads. It starts by accepting a number of threads and initializing two empty lists: ”thread_times” to store the time each thread takes to complete its task, ”threads” to keep track of the created threads. The algorithm then runs a loop for a specified number of threads num_threads, where in each iteration, a new thread is created and tasked with performing a specific function (referred to as task_to_llm). Each thread is given a unique name based on its position in the loop (e.g., ”Thread-1”, ”Thread-2”).

Algorithm 1 Simulating multiple threads to generate results from LLM

Function(num_threads)

1) Initialize two lists:
   - `thread_times` ← [ ] (to store execution times of each thread globally).
   - `threads` ← [ ] (to store all created threads).
2) **For** each i from 0 to `num_threads` - 1:
   - Create a new thread: `thread` ← `CreateThread(target = task_to_llm, name = "Thread-{i + 1}")` where task_to_llm method is passed to execute inside it.
   - Append the new thread to `threads`.
   - Start the thread: `thread.start()`.
3) **For each** thread in `threads`:
   - Wait for the thread to finish: `thread.join()`.

End Function

The thread is then added to the threads list and started immediately. Once all threads have been started, the algorithm enters another loop, where it waits for each thread to complete its task using the join() method. By doing this, the algorithm ensures that all threads are finished before proceeding to the next step. This approach allows for simulating multiple requests happening at the same time, which is useful for testing scenarios like concurrent request handling or evaluating system performance under load, such as in a Denial of Service (DoS) simulation.

We developed Python scripts in order to observe these LLM models by providing the same prompt under any level of requests. While all of the models typically respond well under normal conditions, their response time slows as the load increases, and they eventually almost stop responding under massive request volumes. In our experiment, we used an Apple(M3 Pro) machine having 12 CPU(s) and 18GB of RAM and used Python 3.12.5 and its package manager of pip version 24.2. Since our machine had 12 CPUs, our goal was to evenly distribute the load across all available CPUs. We began our experiment by initiating N requests, where N represents the number of CPUs which was 12 in our case. We progressively increased the number of requests by multiplying N with sequential positive integers, starting with 12, then 24, 36, and so on. This scaling continued until we reached a maximum load of 600 concurrent requests. The entire simulation load can be defined using a mathematical formula:

$$L_n = N \times n \tag{1}$$

In formula 1, N is the CPU count, n is a positive integer that starts at 1 and goes up to 50 serially and Ln is the load level. This means we created 50 levels of DoS load, starting at 12 and ending at 600 as the maximum load as per our machine capacity. In a CSV file, for each level of the DoS attack on all the LLMs, we recorded key features of our analysis. We captured the completion time for each task individually, along with the average time per task and the total time taken for all tasks, while ignoring the overlapping time as the tasks were processed concurrently.

B. Tokenization based DoS attacks

While the earlier sections of this study focused on how LLMs respond to high concurrency in a Denial of Service (DoS) scenario, we further extended our investigation to explore another form of attack: prompt-induced instability using token repetition. This experiment aims to determine whether LLMs can be destabilized without concurrent requests, simply by manipulating the internal structure of a single input prompt. In this extended experiment, we evaluated how LLMs behave when given increasingly long prompts composed of repeated tokens. To ensure a fair comparison with our previous concurrency experiment, we used the exact same base prompt — ”Describe the earth in one easy sentence.”. However, instead of sending it once per request as we did in the multithreading DoS setup, we repeated the same sentence multiple times within a single prompt. For example, the prompt grew into: ”Describe the earth in one easy sentence. Describe the earth in one easy sentence. Describe the earth in one easy sentence...”. Each test progressively increased the number of repetitions, thereby increasing the input token length, while keeping the prompt semantically redundant. We examined how this token

composition alone impacts the LLM's execution time and crash behavior.

VIII. Results and Performance Comparison

A. Concurrent DoS attack

We analyzed several time-based performance metrics and compared them visually. For example, we measured average response time under various levels of load, which is depicted in Figure 6. This figure compares the average response time across three large language models (LLMs): GPT2, BLOOM(bigscience/bloom-560m), and OPT(facebook/opt-1.3b). As the total number of threads increases, all three models demonstrate an increase in average response time. This is expected as higher concurrency levels typically introduce overhead, affecting the response time.

BLOOM shows the slowest performance among the three models. As the number of threads increases, its average response time becomes much longer. This means BLOOM struggles to handle many requests at once, making it the least efficient model in this comparison. When there are far more threads, BLOOM's response time increases significantly. This suggests that BLOOM requires more computational power and resources, which slows it down as the system gets busier.

GPT2 performs better than BLOOM, but it still slows down as more threads are introduced. However, GPT2 handles the increase in threads more gracefully than BLOOM, meaning it's a more efficient model for environments with moderate concurrency. It doesn't slow down as quickly, which makes it a better option in many cases. Compared to BLOOM, GPT2 is more balanced in terms of speed. While it's not the fastest model, it manages workloads better as more threads are added. It's a reliable choice when extremely fast responses are not required, but some level of efficiency in handling multiple threads is still important.

OPT consistently shows the best performance comparatively, with the lowest average response times across all thread levels. Unlike BLOOM and GPT2, OPT manages to keep its response time low even as more threads are added, making it the most efficient model when handling many requests at the same time in this comparison. What makes OPT stand out is its ability to scale well. As the number of threads grows, its response time increases only slightly. This shows that OPT can handle heavy loads without losing efficiency. Its design likely allows it to manage multiple tasks at once better than the other LLM models.

We compared the performance of the LLM(s) under normal and heavy loads. The initial requests were in normal load, and the requests on the last batch were under peak load. The graph in 7 compares the First(initial task) Response Time (dashed lines) and Last(last task) Response Time (solid lines) across the three models: GPT2, BLOOM, and OPT, plotted against the total number of threads.

With these measurements, BLOOM shows the slowest performance in both first and last response times. As the number of threads increases, BLOOM's response times grow significantly, indicating that it struggles to efficiently handle

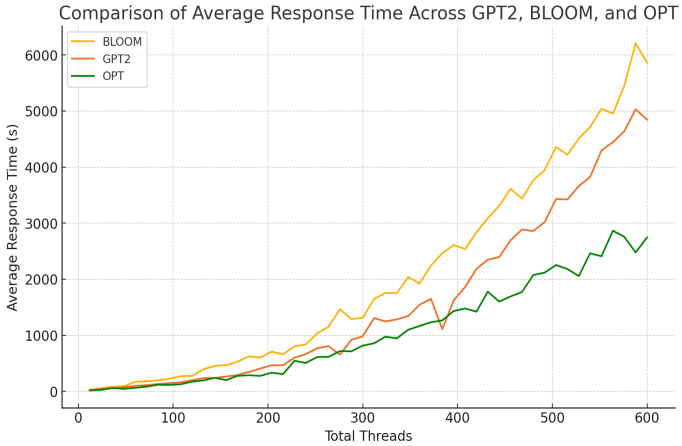

Fig. 6: Comparison of average response time vs total threads

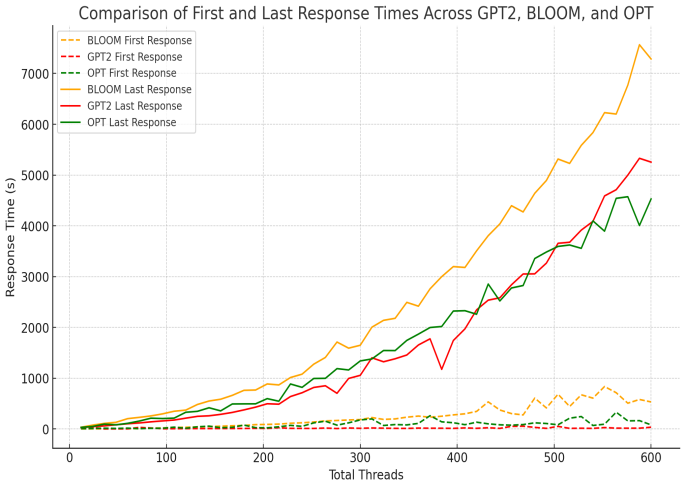

Fig. 7: Comparison of first and last request response time

multiple concurrent requests. The increasing gap between the first and last response times further highlights its inefficiency in managing higher loads.

GPT2 performs moderately better than BLOOM, though it is not as efficient as OPT. It is worth noting that GPT2 performs well on initial tasks across all levels of load. In varying this load, both the first and last response times increase as the number of threads rises, but the rate of increase is more controlled compared to BLOOM. This makes GPT2 a more balanced option, though not optimal for highly concurrent tasks.

OPT shows the best performance maintaining the lowest first and last response times across all thread levels. As concurrency increases, OPT's response times rise very slowly, demonstrating its scalability and efficiency. This model is clearly the most capable of handling multiple requests with minimal performance degradation across all levels of load among the 3 LLM(s).

We also measured the throughput: the number of requests served per minute. The throughput comparison is depicted in Figure 8

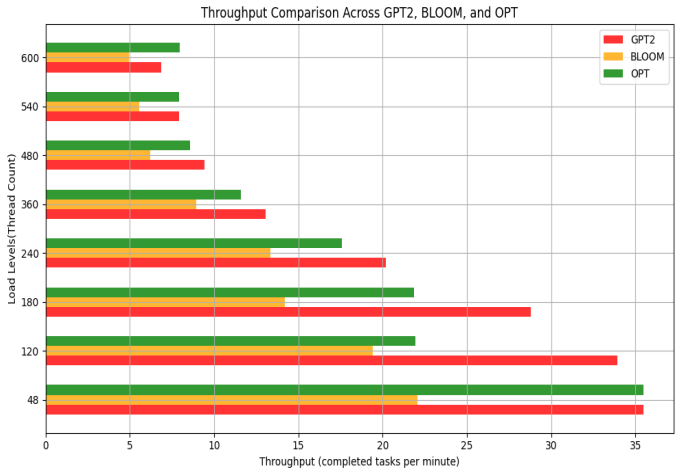

Fig. 8: Throughput comparison

OPT consistently outperforms the other two models, processing requests in a significantly shorter time across almost all levels. For instance, at 600 load level, OPT completes around 8 tasks per minute, compared to GPT2's 7 tasks and BLOOM's 5 tasks per minute. This demonstrates OPT's superior efficiency in handling high-concurrency scenarios.

GPT2 performs better than the other two models when the load is below 500. Although its throughput decreases as the load increases beyond 500, it still outperforms BLOOM. While GPT2 shows noticeable improvement over BLOOM, it falls short of OPT in terms of overall efficiency.

BLOOM shows the lowest throughput across all load levels, indicating inefficiency in handling larger volumes of concurrent requests. At almost all load levels, BLOOM's throughput is around 30-35% lower than the other two. This trend continues as throughput increases, showing that BLOOM struggles to scale under heavy loads. This trend shows that it is exponential growth [43] in general.

Overall, our analysis reveals that while all three models are capable of handling concurrent requests, OPT is the most robust and scalable, maintaining stable performance under increasing load. GPT-2 and Bloom, though effective at lower loads, exhibit significant performance degradation as the number of concurrent requests grows.

B. Token-Based Prompt-Induced DoS Behavior

Similar to our preceding experiments, we tested three Hugging Face-hosted models.

TABLE III: Open source LLM models

LLM Model	Developed By	Parameter Size	Max Token Support	Crash Point
GPT2-XL	OpenAI	1.5 Billions	1024	< 1000
OPT125M	Facebook	125 Millions	2048	< 2048
BLOOM560M	BigScience	560 Millions	2048	> 2048

All models were run sequentially (no concurrency) on the same MacBook Pro M3 Pro system with 12 CPUs and 18 GB RAM to maintain hardware consistency. The behavior of the models under repeated prompt input is visualized in figure 9. Each curve shows how the model response time changes as the number of tokens increases. Crash points are marked with X symbols on the chart, showing where each model stopped functioning or timed out due to internal overload.

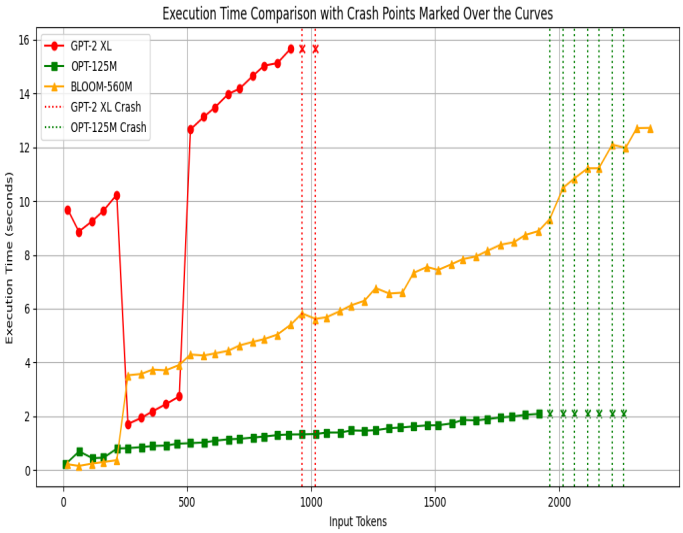

Fig. 9: Response Time Behavior with Repeated Prompts

- GPT-2 XL displayed the most erratic behavior, with response time spiking up and down, and crashing well before its 1024-token limit.
- OPT-125M followed a similar trend, with fluctuations followed by an early crash before their maximum 2048 tokens limit.
- BLOOM-560M, however, remained more stable throughout the experiment. Although it slowed down as the token length increased, it did not crash early but handled longer input more gracefully.

These results are significant because they show that instability was not triggered by the number of requests (as in typical DoS), but by the internal token structure of a single prompt. This suggests that repetitive input, especially repeated sentences, can overload internal processing modules such as attention layers, caching, or decoding mechanisms, even when concurrency is absent. To validate that the instability was due to token repetition and not prompt length alone, we also tested each model with randomly varied natural-language prompts that increased in token size. In contrast to repeated input, the response times for these random prompts grew steadily and predictably, without unexpected spikes or crashes. This contrast behavior underscores the fact that prompt structure and token entropy are critical factors that influence the performance and reliability of LLMs.

To understand whether the instability observed in repeated prompts was caused purely by token length or rather by the nature of the input itself, we conducted a complementary experiment using randomized prompts. These prompts were composed of various natural-language sentences, preserving semantic variation and avoiding any repetition.

For each model—GPT-2 XL, OPT-125M, and BLOOM-560M, we increased the input size by appending more random content to the prompt while measuring the average execution time for each prompt length.

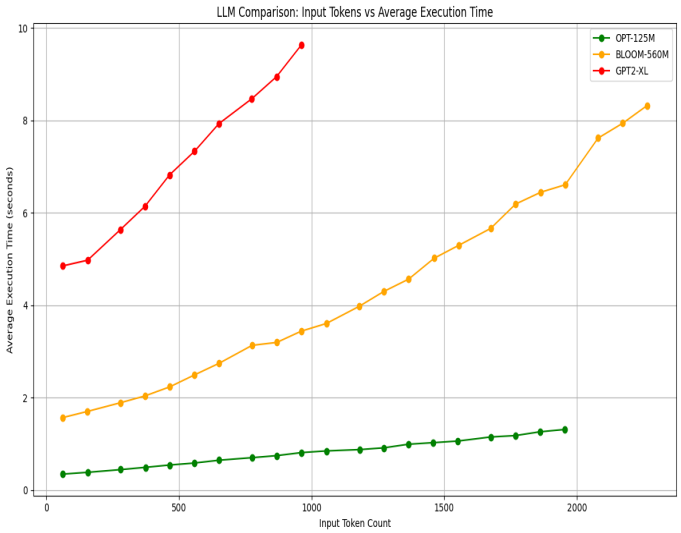

Fig. 10: Average Execution Time with Random Prompts

The results are visualized in figure 10. From the graph, we observe the following patterns:

- GPT-2 XL consistently took the longest time to generate output. Its execution time started around 4.8 seconds at lower token counts and increased sharply to nearly 9.7 seconds as the token count approached 1000. However, the growth was smooth and linear, with no instability or crashes, in contrast to its performance with repeated prompts.
- BLOOM-560M demonstrated moderate execution time, starting from approximately 1.6 seconds and gradually increasing to about 8.3 seconds at 2048 tokens. Its performance showed a clear linear correlation between the length of the input token and the response time. BLOOM handled input increases efficiently, without exhibiting sudden spikes or failures. It is worth noting that the BLOOM model continues to generate output even after reaching its maximum supported context length of 2048 tokens. Instead of crashing, the model degrades gracefully, returning responses that may exhibit lower coherence or truncated content [44]. This behavior suggests that BLOOM handles prompt overflow more robustly compared to models that throw run-time errors or completely halt. However, the decline in output quality beyond the token limit still represents a potential vector for subtle degradation under adversarial load.
- OPT-125M showed the best performance, with the lowest execution times across all token sizes. Its response time grew from around 0.4 seconds to just above 1.3 seconds even at 2000+ tokens. This indicates excellent scalability and response stability under growing prompt sizes.

What's particularly notable is that none of the models crashed early or exhibited execution spikes during this test. The graph literally confirms that token length alone does not cause instability. Instead, it is the structure and repetitiveness of the tokens as observed in the repeated prompt experiment that triggers performance breakdowns.

This experiment validates that: The models' internal mechanisms handle token diversity well, Random prompts with varied language distribute computational load more evenly across model layers (such as attention and decoding), Prompt entropy (diversity) acts as a stabilizing factor, whereas low-entropy inputs (repetitions) can cause resource bottlenecks or memory saturation. In real-world applications, user-generated inputs are typically varied and complex. However, this experiment shows that adversaries can still manipulate model stability by crafting high-token but low-entropy prompts to simulate overload.

This new type of vulnerability, token-based prompt-induced DoS is a stealthier attack vector compared to the traditional concurrency-based DoS attack.

Table IV contrasts the behavioral characteristics of traditional concurrency-based DoS attacks with the Token-Based Prompt-Induced DoS attack introduced in this work. Notably, the latter achieves system degradation without triggering typical defenses such as rate-limiting or thread caps.

TABLE IV: Comparison of Token-Based vs. Concurrency-Based DoS Attacks on LLMs

Criteria	**Concurrency-Based DoS**	**Token-Based Prompt-Induced DoS**
Requires multiple users or threads	Yes	No (single prompt)
Bypasses rate limiting	Often blocked by thresholds	Yes – looks like normal input
Crash reproducibility	Depends on thread scheduler, hard to reproduce exactly	Highly reproducible with fixed and repeated prompt
Exploit complexity	Requires technical knowledge to simulate load	Simple to execute with repeated tokens
Defense mechanism type needed	Infrastructure-level (rate limiters, queues)	Application-level (entropy filters, pattern detectors)
Stealthy (hard to detect)	Not stealthy	Highly stealthy
Prompt engineering required	No (generic input)	Yes (repetition or low entropy)
Impact on system	Slows response under load	Can degrade output or crash model
Triggers infrastructure-level alerts	Likely	Often missed
Works on offline/local LLMs	Not practical (requires network load simulation)	Yes – works even without a network

IX. Comparative Analysis with Existing LLM Defense Mechanisms

While previous works have addressed specific vulnerabilities of LLMs, such as prompt injection and performance bottlenecks, our study focuses uniquely on both concurrency-induced and token-based denial-of-service threats. Table V provides a comparative summary.

TABLE V: Comparison with Existing LLM Defense Frameworks

Feature / Tool	ShieldGPT[29]	SmoothLLM[32]	Sarathi-Serve[45]	Our Work
Primary Goal	Reduce hallucination and improve output safety	Prevent adversarial prompt injections	Improve inference latency and throughput balance	Reveal DoS vulnerabilities through stress tests
Focus Area	Prompt filtering	Prompt smoothing and structure control	Request queuing and adaptive inference	Prompt entropy, concurrency, and architectural stress
Concurrency DoS	✗	✗	Mitigated with load-aware routing	✓Simulated with up to 600 concurrent requests
Token Repetition DoS	✗	✗	✗	✓Novel attack method introduced
Entropy-Aware Filtering	✗	✗	✗	✓Proposed as mitigation
Evaluation	Role templates and prompt alignment	Prompt perturbation and resistance	Latency and throughput plots	Model crash points, token pattern impact
Prompt Structure Consideration	✗	Limited syntactic view	✗	✓Repetition vs. random token structure tested
Prompt Length Scaling	✗	✗	✗	✓Evaluated long repeated prompts vs. random prompts
Crash Point Detection	✗	✗	✗	✓Visualized crash thresholds and instability markers
Thread-Based Simulation	✗	✗	✗	✓Multithreaded Python DoS simulator
Single-Input DoS Attack	✗	✗	✗	✓DoS from a single, repeated prompt tested
Proposed Architectural Mitigation	Prompt templating	Prompt smoothing (Levenshtein)	Queuing system	Entropy-based filtering (future work); Queue-based mitigation (implemented) [18]

X. Limitations and Future Research Direction

In this paper, we tested only two types of DoS attacks on Large Language Models (LLMs): one is flooding the model(concurrent-based DoS), and the other is sending a single long prompt with repeated tokens to overload the model (token-based prompt-induced DoS). Our experiments showed that LLMs struggle under these attacks, but we did not offer any mitigation techniques against the attacks. With the preceding discussions in mind, we offer the following future research directions:

- **Formal Analysis and Prompt Scaling:** We need to employ a formal analysis to justify the discrepancy in performance of the selected models and investigate the performance of the models at varying length of the prompt input.
- **DoS Mitigation:** We have already implemented and validated a queue-based mitigation system to manage concurrency-based DoS attacks [18]. In future work, we plan to design a prompt filtering system capable of detecting DoS attack patterns and triggering appropriate mitigation mechanisms.

XI. Conclusion

Large Language Models (LLMs) have become widely used across various fields due to their ability to handle complex language tasks, but they also possess vulnerabilities that must be addressed. This paper examined two vectors of Denial of Service (DoS) attacks: (1) concurrency-based attacks that overwhelm LLMs through massive parallel requests, and (2) a novel token-based prompt-induced DoS, which destabilizes models using a single low-entropy repeated-token prompt. Our experiments on three open-source LLMs (GPT-2, BLOOM, and OPT) showed that all models experienced performance degradation or crashes, with OPT being the most resilient and BLOOM the least. To address the concurrency-based threat, we validated by implementing a queue-based mitigation approach, as described in our companion work.

Crucially, we demonstrate that token repetition alone—without any concurrency—can trigger failure modes that evade standard rate-limiting defenses, revealing a new class of stealthy, input-level DoS vulnerabilities. These findings highlight the need for robust defenses such as entropy-aware prompt filters and scalable request management systems. As LLMs are increasingly integrated into critical applications, developing resilience against both infrastructure-level and input-based DoS attacks is essential for maintaining their reliability and trustworthiness.

XII. Acknowledgments

The work is supported by the National Science Foundation under Award #2433800, #2421324 and National Institutes of Health Grant $\#5R42LM014356-03$. Any opinions, findings, recommendations, expressed in this material are those of the authors and do not necessarily reflect the views of the NSF and NIH.

References

[1] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, "A comprehensive overview of large language models," *arXiv preprint arXiv:2307.06435*, 2023.

[2] E. Kasneci, K. Seßler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günnemann, E. Hüllermeier *et al.*, "Chatgpt for good? on opportunities and challenges of large language models for education," *Learning and individual differences*, vol. 103, p. 102274, 2023.

[3] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz, and D. Roth, "Recent advances in natural language processing via large pre-trained language models: A survey," *ACM Computing Surveys*, vol. 56, no. 2, pp. 1–40, 2023.

[4] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin *et al.*, "A survey on hallucination in large

language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, 2025.

[5] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," https://owasp.org/www-project-top-10-for-large-language-model-applications/, Nov. 2024, accessed: 2025-06-12.

[6] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng *et al.*, "Prompt injection attack against llm-integrated applications," *arXiv preprint arXiv:2306.05499*, 2023.

[7] M. A. Barek, M. M. Rahman, S. Akter, A. K. I. Riad, M. A. Rahman, H. Shahriar, A. Rahman, and F. Wu, "Mitigating insecure outputs in large language models (llms): A practical educational module," in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2024, pp. 2424–2429.

[8] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications - 2025 Edition," https://owasp.org/www-project-top-10-for-large-language-model-applications/, Jan. 2025, accessed: 2025-06-12.

[9] D. Bowen, B. Murphy, W. Cai, D. Khachaturov, A. Gleave, and K. Pelrine, "Scaling trends for data poisoning in llms," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 26, 2025, pp. 27 206–27 214.

[10] S. Wang, Y. Zhao, Z. Liu, Q. Zou, and H. Wang, "Sok: Understanding vulnerabilities in the large language model supply chain," *arXiv preprint arXiv:2502.12497*, 2025.

[11] U. Iqbal, T. Kohno, and F. Roesner, "Llm platform security: applying a systematic evaluation framework to openai's chatgpt plugins," in *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*, vol. 7, 2024, pp. 611–623.

[12] OWASP Foundation, "Llm08: Excessive agency – owasp top 10 for llms," 2024, accessed: 2024-04-16. [Online]. Available: https://genai.owasp.org/llmrisk2023-24/llm08-excessive-agency/

[13] J. Y. Bo, S. Wan, and A. Anderson, "To rely or not to rely? evaluating interventions for appropriate reliance on large language models," *arXiv preprint arXiv:2412.15584*, 2024.

[14] A. Liu and A. Moitra, "Model stealing for any low-rank language model," *arXiv preprint arXiv:2411.07536*, 2024.

[15] R. Sood, "Cyber forensic manual for denial of service attack (dos attack)," *Available at SSRN 4660684*, 2023.

[16] F. Lau, S. H. Rubin, M. H. Smith, and L. Trajkovic, "Distributed denial of service attacks," in *Smc 2000 conference proceedings. 2000 ieee international conference on systems, man and cybernetics.'cybernetics evolving to systems, humans, organizations, and their complex interactions'(cat. no. 0*, vol. 3. IEEE, 2000, pp. 2275–2280.

[17] J. Mölsä, "Effectiveness of rate-limiting in mitigating flooding dos attacks." in *Communications, Internet, and Information Technology*. Citeseer, 2004, pp. 155–160.

[18] M. A. Barek, M. Bajlur Rashid, M. M. Rahman, A. Kamrul Islamc Riad, G. Francia, H. Shahriar, and S. I. Ahamed, "Vulnerability to stability: Scalable large language model in queue-based web service," in *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2025, pp. 995–1000.

[19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[20] B. Groza and M. Minea, "Formal modelling and automatic detection of resource exhaustion attacks," in *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security*, 2011, pp. 326–333.

[21] S. Sabri, N. Ismail, and A. Hazzim, "Slowloris dos attack based simulation," in *IOP Conference series: materials science and engineering*, vol. 1062, no. 1. IOP Publishing, 2021, p. 012029.

[22] S. Rossi, A. M. Michel, R. R. Mukkamala, and J. B. Thatcher, "An early categorization of prompt injection attacks on large language models," *arXiv preprint arXiv:2402.00898*, 2024.

[23] M. Guastalla, Y. Li, A. Hekmati, and B. Krishnamachari, "Application of large language models to ddos attack detection," in *International Conference on Security and Privacy in Cyber-Physical Systems and Smart Vehicles*. Springer, 2023, pp. 83–99.

[24] T. Kudo and J. Richardson, "Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," *arXiv preprint arXiv:1808.06226*, 2018.

[25] N. Rajaraman, J. Jiao, and K. Ramchandran, "Toward a theory of tokenization in llms," *arXiv preprint arXiv:2404.08335*, 2024.

[26] D. Wang, Y. Li, J. Jiang, Z. Ding, G. Jiang, J. Liang, and D. Yang, "Tokenization matters! degrading large language models through challenging their tokenization," *arXiv preprint arXiv:2405.17067*, 2024.

[27] A. Ben-Artzy and R. Schwartz, "Attend first, consolidate later: On the importance of attention in different llm layers," *arXiv preprint arXiv:2409.03621*, 2024.

[28] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen, "A survey on the memory mechanism of large language model based agents," *arXiv preprint arXiv:2404.13501*, 2024.

[29] T. Wang, X. Xie, L. Zhang, C. Wang, L. Zhang, and Y. Cui, "Shieldgpt: An llm-based framework for ddos mitigation," in *Proceedings of the 8th Asia-Pacific Workshop on Networking*, 2024, pp. 108–114.

[30] J. Mirkovic and P. Reiher, "A taxonomy of ddos attack and ddos defense mechanisms," *ACM SIGCOMM Computer Communication Review*, vol. 34, no. 2, pp. 39–53, 2004.

[31] J. Nazario, "Ddos attack evolution," *Network Security*, vol. 2008, no. 7, pp. 7–10, 2008.

[32] A. Robey, E. Wong, H. Hassani, and G. J. Pappas, "Smoothllm: Defending large language models against jailbreaking attacks," *arXiv preprint arXiv:2310.03684*, 2023.

[33] A. K. I. Riad, M. A. Barek, M. M. Rahman, M. S. Akter, T. Islam, M. A. Rahman, M. R. Mia, H. Shahriar, F. Wu, and S. I. Ahamed, "Enhancing hipaa compliance in ai-driven mhealth devices security and privacy," in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2024, pp. 2430–2435.

[34] M. A. Rahman, M. Abdul Barek, A. K. Islam Riad, M. Mostafizur Rahman, M. B. Rashid, M. Raihan Mia, H. Shahriar, G. Francia, F. Wu, A. Cuzzocrea, and S. I. Ahamed, "Embedding with large language models for classification of hipaa safeguard compliance rules," in *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2025, pp. 1040–1046.

[35] M. B. Rashid, M. A. Barek, M. M. Rahman, S. Yeasmin, H. Shahriar, and S. I. Ahamed, "Secure ios mhealth apps development: An ide-embedded framework for hipaa-aware coding," in *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2025, pp. 1101–1107.

[36] M. R. Mia, H. Shahriar, M. Valero, N. Sakib, B. Saha, M. A. Barek, M. J. H. Faruk, B. Goodman, R. A. Khan, and S. I. Ahamed, "A comparative study on hipaa technical safeguards assessment of android mhealth applications," *Smart Health*, vol. 26, p. 100349, 2022.

[37] K. I. Riad, S. Ahmed, M. Z. Nizum, M. A. Barek, M. B. Rashid, M. Mostafizur Rahman, G. Francia, and H. Shahriar, "Privacy-preserving self-supervised learning for secure image processing: A byol-based framework for mnist and chest x-ray data," in *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2025, pp. 1136–1145.

[38] M. M. Rahman, M. A. Barek, M. S. Akter, A. K. Islam Riad, M. A. Rahman, H. Shahriar, A. Rahman, and F. Wu, "Authentic learning on devops security with labware: Git hooks to facilitate automated security static analysis," in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2024, pp. 2418–2423.

[39] J. Yang, "Rethinking tokenization: Crafting better tokenizers for large language models," *International Journal of Chinese Linguistics*, vol. 11, no. 1, pp. 94–109, 2024.

[40] T. Le Scao, A. Fan, C. Akiki, E. Pavlick, S. Ilić, D. Hesslow, R. Castagné, A. S. Luccioni, F. Yvon, M. Gallé *et al.*, "Bloom: A 176b-parameter open-access multilingual language model," 2023.

[41] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, "Opt: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.

[42] Z. A. Aziz, D. N. Abdulqader, A. B. Sallow, and H. K. Omer, "Python parallel processing and multiprocessing: A rivew," *Academic Journal of Nawroz University*, vol. 10, no. 3, pp. 345–354, 2021.

[43] J. Tague, J. Beheshti, and L. Rees-Potter, "The law of exponential growth: evidence, implications and forecasts," 1981.

[44] T. L. Scao, A. Fan, C. Akiki, E. Pavlick *et al.*, "Bloom: A 176b-parameter open-access multilingual language model," *arXiv preprint arXiv:2211.05100*, 2022.

[45] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," *arXiv preprint arXiv:2403.02310*, 2024.